강남대학교 / 게임프로그래밍

Unity 기초와 게임 프로그래밍

중간고사 개념 복습

2026학년도 2학기 · 담당교수 김영호

복습할 기초 개념

영역	복습할 내용
Unity 기본 기능	씬, 오브젝트, 좌표, Transform, 컴포넌트, Prefab
물리·충돌·입력	시간과 이동, Rigidbody2D, Collider2D, Action과 Binding
표현과 코드 구조	애니메이션, 카메라, UI, 리팩토링, 객체 재사용
C# 기초	변수, 대입, 조건, 반복, bool, 함수 호출과 return

이 자료의 기초 개념과 코드 흐름이 시험 범위입니다.

프로젝트별 규칙·이름·구현 순서·Inspector 수치는 묻지 않습니다. 데이터 저장, 세부 입력 API와 리바인딩 구현도 제외합니다.

씬에 오브젝트를 배치하고 컴포넌트로 기능을 붙입니다

구성	뜻
Scene	오브젝트를 배치하고 구성해 둔 공간
GameObject	씬에 있는 개별 오브젝트
Component	오브젝트의 위치, 표시, 물리 등 기능을 담당
Script	직접 작성한 동작과 규칙을 코드로 표현

오브젝트의 이름만으로 기능이 정해지지는 않습니다. 붙어 있는 컴포넌트와 코드의 동작을 봐야 합니다.

Hierarchy는 씬의 오브젝트와 계층 관계를 보여 주고 Inspector는 선택한 대상의 컴포넌트와 설정을 보여 줍니다.

위치, 회전, 크기를 각각 조절합니다

값	역할	바꾸면
Position	오브젝트의 위치	배치가 이동함
Rotation	오브젝트의 회전	방향이 바뀜
Scale	각 축의 크기 비율	모양이 커지거나 작아짐

기본 월드 좌표에서 X는 좌우, Y는 위아래, Z는 앞뒤 축입니다. Vector3는 세 축의 값을 함께 표현합니다.

Scale은 실제 길이가 아니라 크기 비율입니다. 크기를 줄여도 Position 값이 자동으로 바뀌지는 않습니다. 최종 크기는 원본과 부모 Transform의 영향도 받습니다.

좌표를 읽을 때는 기준과 축을 함께 봅니다

구분	표현
Vector2	두 값으로 표현: x, y
Vector3	세 값으로 표현: x, y, z
월드 좌표 / position	월드 원점을 기준으로 한 위치
로컬 좌표 / localPosition	부모 Transform을 기준으로 한 위치

Vector2와 Vector3는 위치뿐 아니라 방향이나 다른 값도 표현할 수 있습니다. 코드에서 어떻게 사용하느냐에 따라 값의 의미가 달라집니다.

같은 축에서 위치 1과 5의 차이는 4입니다. 반대로 빼면 −4지만 두 위치 사이의 거리는 같습니다. 벡터 타입의 세부 API는 보충 설명입니다.

부모를 움직이면 자식도 함께 움직입니다

변경	부모	자식
부모를 이동·회전	위치·회전이 바뀜	부모의 변화에 영향을 받음
자식만 이동·회전	그대로 유지	부모를 기준으로 위치·회전이 바뀜

localPosition은 부모를 기준으로 한 위치이고 position은 월드 좌표의 위치입니다.

자식의 그림만 회전하면 부모에 붙은 충돌 영역은 함께 회전하지 않습니다. 어떤 Transform을 바꾸느냐에 따라 영향을 받는 대상이 달라집니다.

컴포넌트마다 맡는 역할이 다릅니다

컴포넌트	맡은 역할	설정을 바꾸면
SpriteRenderer	2D 스프라이트 표시	그림의 표시 여부가 달라짐
Rigidbody2D	2D 물리 움직임	중력과 물리 동작이 달라짐
Collider2D	충돌·통과를 감지할 영역	접촉 영역과 감지 방식이 달라짐
Transform	위치, 회전, 크기	배치와 모양의 크기가 달라짐

GameObject에 붙은 컴포넌트들이 각자의 기능을 처리합니다. 하나의 오브젝트에도 여러 컴포넌트가 함께 있을 수 있습니다.

그림이 보이지 않아도 물리 동작은 계속될 수 있습니다.

GetComponent는 붙어 있는 컴포넌트를 가져옵니다

```
Rigidbody2D body = GetComponent<Rigidbody2D>();
```

이 코드는 같은 GameObject에 붙은 Rigidbody2D를 body로 참조합니다. body로 해당 컴포넌트의 값과 기능을 사용할 수 있습니다.

구분	의미
컴포넌트 추가	오브젝트에 기능을 새로 붙임
GetComponent	이미 붙어 있는 컴포넌트를 가져옴

해당 컴포넌트가 없으면 null을 반환합니다. 기본 GetComponent는 자식이나 다른 오브젝트까지 검색하지 않습니다. 호출 문법은 보충 설명이며 이번 시험에서 묻지 않습니다.

참조는 객체를 가리키고 null은 대상이 없는 상태입니다

```
Rigidbody2D body = GetComponent<Rigidbody2D>();
if (body == null)
    return;
```

body는 Rigidbody2D 컴포넌트를 가리키는 변수입니다. 참조를 가져온다고 그 컴포넌트가 복제되지는 않습니다.

결과	의미
컴포넌트를 찾음	body로 그 컴포넌트에 접근 가능
찾지 못해 null 반환	사용할 대상이 없으므로 확인 필요

대상 없이 멤버에 접근하면 오류가 날 수 있습니다. 이 예시는 대상이 없으면 이후 처리를 하지 않습니다. 세부 예외 종류는 보충 설명입니다.

오브젝트와 컴포넌트의 비활성화는 다릅니다

작업	영향
gameObject.SetActive(false)	오브젝트와 그 아래 자식의 활성 동작을 중지
spriteRenderer.enabled = false	그림 표시만 중지. 다른 컴포넌트는 계속 동작할 수 있음
필드 초기화	변수에 시작 값을 직접 대입

비활성화는 객체를 삭제하거나 필드 값을 초기화하는 일이 아닙니다. 다시 사용할 때도 이전 값이 남을 수 있습니다.

오브젝트 전체, 특정 컴포넌트, 변수의 값을 구분합니다.

Prefab을 Instantiate하면 새 객체가 생깁니다

Prefab 에셋

반복 사용할 오브젝트의 구성을 보관합니다.
컴포넌트와 초기 설정을 함께 묶습니다.

씬의 인스턴스

원본을 바탕으로 만들어진 실제 객체입니다.
인스턴스마다 위치와 상태를 따로 가집니다.

Prefab 원본	Instantiate	새 인스턴스

Instantiate는 기존 객체를 검색하는 기능이 아닙니다.

초기화, 프레임 처리, 물리 처리를 구분합니다

함수	호출 기준과 기본 역할
Awake	인스턴스 초기화 때 참조와 내부 값 준비
Start	활성 스크립트의 첫 Update 전에 한 번 실행
Update	활성화된 동안 매 프레임 실행
FixedUpdate	물리 업데이트의 일정한 시간 간격으로 실행

Update는 화면 프레임마다 실행되고 FixedUpdate는 물리 시간 간격에 맞춰 실행됩니다. 따라서 FixedUpdate의 실행 횟수와 화면 프레임 수가 항상 같지는 않습니다.

스크립트 사이의 세부 실행 순서는 시험에서 묻지 않습니다. FixedUpdate의 세부 시간 설정은 보충 설명입니다.

속도에 지난 시간을 곱해 이동량을 구합니다

```
transform.position += Vector3.right * speed * Time.deltaTime;
```

비교	프레임마다 1만큼 이동	지난 시간을 곱해 이동
낮은 프레임 수	1초 동안 실행 횟수가 적음	각 프레임의 시간 간격이 큼
높은 프레임 수	1초 동안 실행 횟수가 많음	각 프레임의 시간 간격이 작음

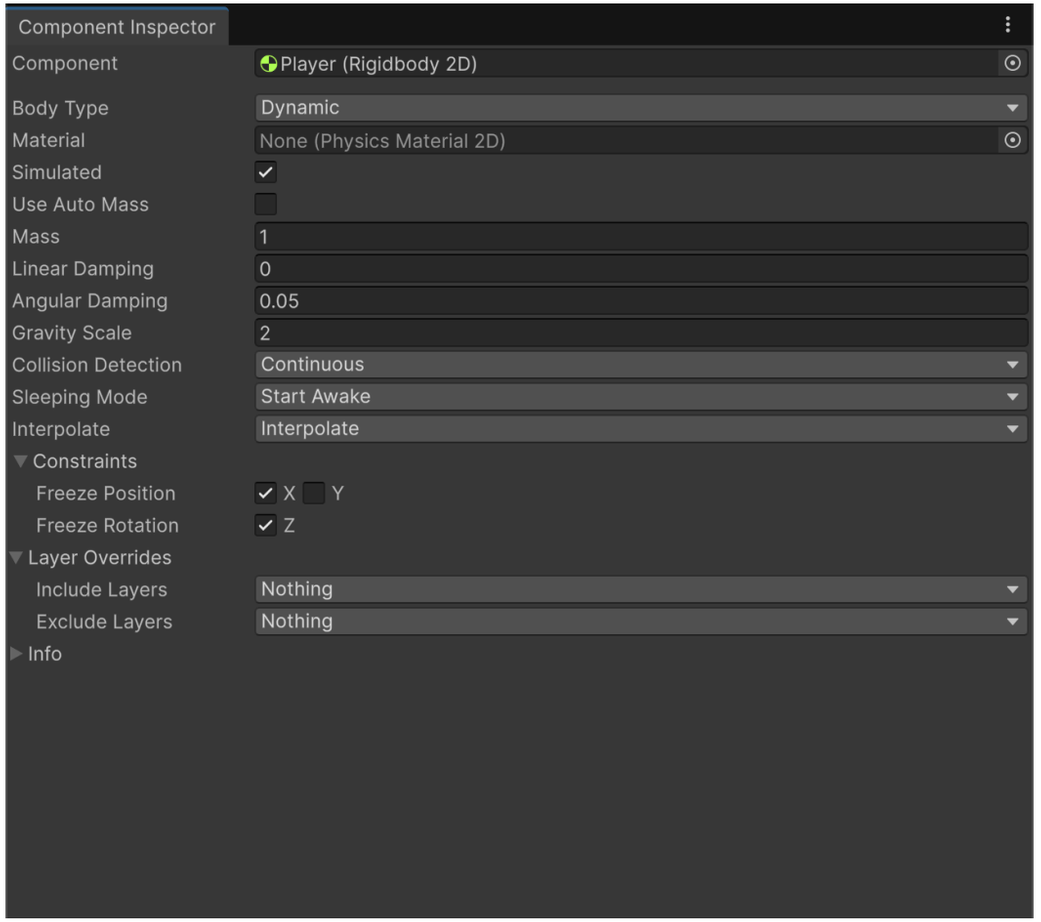
프레임마다 고정 거리를 더하면 프레임 수에 따라 1초 이동 거리가 달라집니다.

Gravity Scale은 중력의 영향을 조절합니다

Dynamic Rigidbody2D

중력과 힘의 영향을 받는 물리 움직임에 사용합니다.
중력이 아래로 작용할 때 Gravity Scale을 높이면 아래쪽 가속이 커집니다.

시작 위치나 애니메이션 속도를 조절하는 설정은 아닙니다.
다른 물리 설정은 같다고 가정합니다.



Rigidbody2D의 Gravity Scale 설정 예시

Collider와 Trigger는 사용 목적이 다릅니다

물리적으로 막는 영역

오브젝트의 통과를 막는 바닥이나 벽에 사용합니다.
물리 설정에 따라 접촉과 충돌 반응을 처리합니다.

통과를 감지하는 영역

점수 구간이나 도착 영역에서 오브젝트가 지나가는지 감지합니다.
Collider의 Is Trigger를 사용합니다.

통과시키려고 Collider를 없애면 감지할 영역도 사라집니다.

이벤트를 받으려면 Rigidbody, 레이어 등 다른 물리 조건도 올바르게 구성되어 있어야 합니다.

충돌 영역과 물리 동작, 이벤트를 구분합니다

구성	역할
Collider2D	접촉하거나 겹치는 영역을 정의
Rigidbody2D	물리 동작과 물리 상호작용에 참여
Collision 이벤트	Trigger가 아닌 충돌의 접촉을 알림
Trigger 이벤트	물리적으로 막지 않는 영역의 겹침을 알림

처음 접촉할 때는 OnCollisionEnter2D, Trigger 영역에 들어갈 때는 OnTriggerEnter2D를 사용합니다.

Trigger 이벤트에는 최소 한쪽에 Rigidbody2D가 필요합니다. Body Type과 레이어 설정도 결과에 영향을 줍니다.
2D와 3D 물리는 서로 다른 시스템입니다. 세부 이벤트 문법은 보충 설명입니다.

게임 행동과 장치의 입력을 나눕니다

항목	역할과 특징
Input Action	게임에서 받을 행동이나 값
Binding	그 행동에 연결할 키·버튼 등의 입력
하나의 Action에 여러 Binding	각 입력으로 같은 행동을 요청할 수 있음
같은 실행 함수로 연결	장치별로 게임 동작 코드를 중복 작성하지 않음

예를 들어 키보드의 키와 게임패드의 버튼을 같은 Action에 연결하면, 두 장치에서 같은 행동을 요청할 수 있습니다.

누른 순간과 누르고 있는 동안의 차이

입력 기준	뜻	사용 예
누른 순간	키를 누르기 시작했을 때 한 번 감지	한 번 누르면 한 번 점프
누르고 있는 동안	키가 눌린 상태를 계속 확인	방향키를 누르는 동안 이동

누른 순간을 감지하면 키를 계속 누르고 있어도 새로 누르기 전까지 반복 실행하지 않습니다. 눌린 상태를 매 프레임 확인하면 키를 누르는 동안 계속 실행할 수 있습니다.

Input System의 세부 이벤트 이름이나 설정 문법은 시험에서 묻지 않습니다.

Clip, Controller, Animator의 역할

구성	역할	예
Animation Clip	시간에 따른 속성 변화를 기록한 에셋	걷기 동작, Sprite 교체
Animator Controller	상태와 상태 사이의 전환을 정의한 에셋	대기에서 이동으로 전환
Animator	Controller에 따라 애니메이션을 재생하는 컴포넌트	오브젝트에서 현재 상태의 동작 재생

Animator에 Controller를 연결하면 Controller의 현재 상태에 지정된 Clip을 재생합니다. Clip에는 재생할 내용이 담겨 있고 Controller는 재생 흐름을 정합니다.

같은 Clip을 반복 재생할지는 Clip의 Loop Time 설정으로 정합니다.

Parameter 값으로 전환 조건을 판단합니다

개념	뜻	예
State	현재 재생할 동작을 나타내는 상태	Idle, Move
Transition	한 상태에서 다른 상태로 넘어가는 연결	Idle에서 Move로 전환
Parameter	전환 조건 등에 사용하는 값	bool 타입의 IsMoving

animator.SetBool("IsMoving", true)로 값을 바꿉니다. Controller는 IsMoving = true와 나머지 조건을 확인한 뒤 모두 충족하면 Idle에서 Move로 전환합니다.

IsMoving은 직접 정한 이름입니다. Controller에 Parameter와 전환 조건이 정의되어 있어야 합니다. API 문법과 세부 설정은 시험에서 묻지 않습니다.

카메라는 월드의 어느 영역을 보여 줄지 정합니다

카메라의 위치와 회전으로 어디서 어느 방향을 볼지 정합니다. Projection은 월드를 화면에 표현하는 방식입니다.

설정	역할
Perspective	같은 물체도 멀리 있으면 작게 보임
Orthographic	같은 물체는 거리가 달라도 같은 크기로 보임
Orthographic Size	직교 카메라가 보는 세로 범위의 절반

같은 화면 비율에서 Size를 키우면 더 넓은 영역이 들어오므로 물체는 화면에서 작게 보입니다.

Perspective의 화각은 Field of View로 조절합니다. 수치나 화각 계산은 시험에서 묻지 않습니다.

UI는 Canvas 아래에 구성합니다

구성	역할
Canvas	UI를 그리는 기준
RectTransform	UI 요소의 위치와 크기, 앵커 등을 관리
Image / TextMeshPro	이미지 / 글자를 표시
Button	사용자의 클릭을 받는 UI 컴포넌트

Button의 On Click 이벤트는 등록된 대상의 함수를 호출합니다. 버튼 이름과 함수 이름이 같아도 자동으로 연결되지는 않습니다.

여기서는 Canvas 기반 UI를 설명합니다. UI 표시와 다른 스크립트의 실행 여부는 별개입니다.

리팩토링과 최적화는 목적이 다릅니다

개념	목적
리팩토링	외부 동작을 유지하면서 코드의 내부 구조를 개선
책임 분리	서로 다른 역할의 코드를 구분해 수정 범위를 줄임
중복 제거	같은 처리나 규칙을 여러 곳에서 따로 수정하지 않도록 정리
최적화	실행 시간이나 메모리 등의 비용을 줄임

코드를 읽기 쉽게 정리했다고 실행 속도까지 빨라지는 것은 아닙니다. 성능이 나아졌는지는 측정으로 확인합니다.

생성과 삭제 대신 같은 객체를 다시 사용합니다

오브젝트 풀은 인스턴스를 보관했다가 다시 사용하는 방식입니다. 생성과 삭제가 반복될 때 드는 비용을 줄입니다.

작업	의미
Instantiate	새 인스턴스를 만듦
Destroy	객체를 삭제함
비활성화 후 재사용	기존 인스턴스를 보관했다가 다시 사용함

풀의 용량과 객체 수명에 따라 새로 생성하거나 일부를 삭제하기도 합니다. 핵심은 같은 객체를 반복해서 쓰는 데 있습니다.

변수는 값을 보관하고 타입은 값의 종류를 정합니다

타입	보관할 값	예
int	정수	3
float	실수	0.5f
bool	참 또는 거짓	true / false
string	문자열	"Hello"

int value = 3;에서 int는 타입, value는 변수 이름, 3은 초기값입니다. 변수 이름은 직접 정합니다.

실행 중 대입문으로 값을 바꿀 수 있습니다. 뒤의 예시는 int와 bool을 중심으로 읽습니다. 다른 타입의 세부 문법은 보충 설명입니다.

변수의 값은 실행 순서대로 바뀝니다

```
int number = 3;
number += 4;
number = 2;
number++;
```

값의 변화

1. 처음 값은 3입니다.
2. 4를 더해 7이 됩니다.
3. 2를 대입하면 7을 버리고 2가 됩니다.
4. 1을 더해 최종 값은 3입니다.

대입은 기존 값에 더하는 것이 아니라 값을 바꾸는 작업입니다.

대입은 값을 바꾸고 비교는 참·거짓을 구합니다

연산자	의미	예
=	오른쪽 값을 대입	value = 3
== / !=	같다 / 다르다	3 == 3 → true
> / <	크다 / 작다	3 > 5 → false
>= / <=	크거나 같다 / 작거나 같다	3 <= 3 → true

value = 3;은 값을 바꾸는 문장입니다. value == 3은 값이 3인지 비교하는 식입니다.

비교 결과는 bool입니다. 조건문은 이 참·거짓 결과로 실행 여부를 판단합니다.

여러 조건을 묶거나 참·거짓을 뒤집습니다

연산자	의미	a = true, b = false일 때
&&	두 조건이 모두 참이면 참	a && b → false
||	하나 이상의 조건이 참이면 참	a || b → true
!	참·거짓을 반대로 바꿈	!a → false

value > 0 && value < 10은 value가 0보다 크고 10보다 작을 때 참입니다.

&&와 ||는 왼쪽 결과만으로 판단할 수 있으면 오른쪽 식을 평가하지 않습니다. 복잡한 식은 괄호로 묶어 읽습니다.

조건이 참일 때만 안의 코드를 실행합니다

```
int value = 9;
int highest = 12;
if (value > highest)
    highest = value;

value = 15;
if (value > highest)
    highest = value;
```

첫 번째 비교

9는 12보다 크지 않으므로 대입하지 않습니다. highest는 12입니다.

두 번째 비교

value가 15로 바뀌었습니다. 15는 12보다 크므로 highest에 15를 대입합니다.

같은 조건식이라도 검사할 때의 변수 값이 바뀌면 실행 결과가 달라집니다.

시작 값과 종료 조건으로 횟수를 셉니다

```
int total = 0;
for (int i = 0; i < 4; i++)
{
    total += 2;
}
```

i	0	1	2	3
실행 후 total	2	4	6	8

i가 4가 되면 조건이 거짓이어서 반복이 끝납니다. 반복 횟수는 네 번이고, total은 8입니다.

함수는 호출할 때 실행됩니다

```
int value = 1;

void SetValue()
{
    value = 3;
}
```

호출 전

함수를 선언했어도 value는 아직 1입니다.

SetValue 호출 후

함수 안의 대입이 실행되어 value는 3입니다.

다시 호출할 때도 같은 대입이 실행됩니다. 값을 더하는 코드가 아니므로 6이 되지는 않습니다.

매개변수는 함수가 전달받은 값을 담는 변수입니다

```
void PrintNumber(int number)
{
    Debug.Log(number);
}
```

구분	뜻
매개변수 / int number	함수가 값을 전달받을 자리
인수 / 3	PrintNumber(3)을 호출할 때 전달하는 값

PrintNumber(3);을 호출하면 함수 안의 number에 3이 전달됩니다. PrintNumber(7);을 호출하면 7이 전달됩니다.

int처럼 값을 전달하는 기본 예시입니다. ref, out과 참조 타입의 전달 방식은 다루지 않습니다.

반환값은 함수를 호출한 쪽에 돌려주는 값입니다

```
int Add(int a, int b)
{
    return a + b;
}
```

int result = Add(2, 3);에서 반환한 5가 result에 대입됩니다.

선언	의미
int Add(...)	int 값을 반환하는 함수
void PrintNumber(...)	반환값이 없는 함수

return 값;은 값을 돌려주고 함수를 끝냅니다. void 함수의 return;은 값 없이 함수를 끝냅니다.

bool 조건과 return의 역할

```
bool processed = false;
int count = 4;

void Process()
{
    if (processed) return;
    count += 2;
    processed = true;
}
```

두 번 호출하면

첫 호출에서는 false이므로 count에 2를 더하고 processed를 true로 바꿉니다. count는 6입니다.
두 번째 호출에서는 true이므로 return에서 끝납니다. count는 6을 유지합니다.

필드 값은 함수가 끝나도 다음 호출에 남아 있다고 가정합니다.

Unity 공식 문서

- MonoBehaviour.Update
- Time.deltaTime
- Pooling and reusing objects
- Input System / Actions

참고 문서에는 시험 범위 밖의 세부 API도 포함됩니다. 공부 범위는 이 강의자료와 시험범위 안내를 기준으로 합니다.
이어서: 데이터 처리와 저장·읽기